

Intro to CLIs

Learning Goals

- Use your Python skills to build an interactive command-line application.
 - Follow best practices in CLI design in Python.
-

Key Vocab

- **Command Line:** a text-based interface that is built into your computer's operating system. It allows you to access the files and applications on your computer manually or through scripts.
 - **Terminal:** the application in Mac OS that allows you to access the command line.
 - **Command Shell/Powershell:** the applications in Windows that allow you to access the command line.
 - **Command-Line Interface (CLI):** a text-based interface used to run programs, manage files and interact with objects in memory. As the name suggests, it is run from the command line.
-

Introduction

Now that we're in the home stretch of Phase 3, it's time to prepare for your project. You will see the rubric for the project in the next page in this module, but before we get there, let's take some time to explore the format for the project: a **command-line interface** (or **CLI**).

A CLI is a program that is run from the command-line and provides a text-based interface to allow you to carry out various tasks. While the functionality of CLIs is limited by the capabilities of Terminal (in Mac OS) and Command Shell/Powershell (in Windows), *everything you have learned in Phase 3 can be carried out through a CLI.*

Building a Simple CLI

Before we dive into best practices for Python CLIs, let's build a very simple CLI. Fork and clone this lesson from GitHub and open up `grade_reports.py` from the `lib/` directory to follow along.

The first thing that we need to do is scaffold our CLI so that it can be run from the command line. To do this, we'll need to create a script. There are two pieces we know are going into the script regardless of the particulars: the **shebang** and our `if __name__ == '__main__':` block:

```
#!/usr/bin/env python3

if __name__ == '__main__':
    pass
```

Remember that the shebang tells the command line that this program should be executed using the Python 3 interpreter. We can technically still run this as a script without it, but that would require us to write `python grade_reports.py` every time we wanted to do so. Since other programmers using your CLI might not know about this requirement, you should always include the shebang. In the `lib/` directory, run `chmod +x grade_reports.py` to make your script executable.

The `if __name__ == '__main__':` block tells the interpreter that this script should only be run if `./grade_reports.py` itself is being called from the command line. This is important if you want to import any objects from `grade_reports.py` into other modules- if you don't include this code block, the full script will be run whenever your other module runs the import. That's not likely to be a helpful feature in your CLI.

Let's start to add some very simple functionality to our CLI. We want to produce a grade report a full class of students; we're going to use Python's `input()` and `open()` functions to do so.

```
#!/usr/bin/env python3

def create_grade_report(student_grades):
    with open('reports/grade_report.txt', 'w') as gr:
        gr.write(student_grades)

if __name__ == '__main__':
```

```
student_grades = input("Student name, grade: ")
create_grade_report(student_grades)
```

Run `./grade_reports.py` to execute your script:

Student name, grade: Ben, F

Check the `reports` directory- you should see that `grade_report.txt` has been generated and contains your input! Still, this is not really useful for generating grade reports. Let's use a `while` loop to continue collecting student grades until there are no more to enter and a `for` loop to write the grades line-by-line:

```
#!/usr/bin/env python3

def create_grade_report(student_grades):
    with open('./reports/grade_report.txt', 'w') as gr:
        for grade in student_grades:
            # add '\n' to write grades on separate lines
            gr.write(grade + '\n')

if __name__ == '__main__':
    student_grades = []

    grade = input("Student name, grade: ")
    while grade:
        student_grades.append(grade)
        # end when no grade is entered
        grade = input("Student name, grade: ")

    create_grade_report(student_grades)
```

► Which mode would we set `open()` to if we wanted to update `grade_report.txt`, rather than overwrite it?

Now let's enter a classroom's worth of grades:

```
Student name, grade: Ben, F
Student name, grade: Prabhdip, A
Student name, grade: Alvee, A
Student name, grade: Jeff, B
Student name, grade: Jerrica, A
Student name, grade: Gustave, F
Student name, grade: Katie, A
Student name, grade:          # hit enter to complete
```

Check the `reports/` directory again- now you should see that `grade_report.txt` has been regenerated and contains grades for all of your students!

There are still, of course, many ways to improve this CLI. The instructions for the end user could certainly be much clearer. Maybe you want to be able to toggle between updating and generating new grade reports- `argparse` [↗\(https://docs.python.org/3/library/argparse.html\)](https://docs.python.org/3/library/argparse.html) would be helpful for that. Maybe you want timestamps for your grade reports to help you keep them organized- `datetime` [↗\(https://docs.python.org/3/library/datetime.html\)](https://docs.python.org/3/library/datetime.html) would be very helpful there. Maybe you're (rightfully) biased against `.txt` files and want to keep your data in a spreadsheet- `csv` [↗\(https://docs.python.org/3/library/csv.html\)](https://docs.python.org/3/library/csv.html) would be the right module to import there.

Best Practices in CLI Design

Building a simple CLI doesn't take up too much code, but when you start to include a wide range of functionality in a single CLI, it can start to get messy. To make your life easier, keep a few things in mind as you work on your Phase 3 project:

Separate User Input from Functionality

A CLI depends on user input and a lot of code to act on it, but that doesn't mean it all has to be in the same place. As with any other Python program, your CLI should be grouped into classes and functions. Your scripted code- that is, the code inside of your `if __name__ == '__main__':` block- should *only include user input and calls to classes and functions*.

Let's take a look:

```
#!/usr/bin/env python3

class MyClass:
    def __init__(self, user_input):
        self.value = user_input

def my_function(my_object):
    # returns a final value for the CLI workflow

if __name__ == '__main__':
    user_input = input("Enter something here: ")
    my_object = MyClass(user_input)
    print(my_function(my_object))
```

Here, we've factored most of our code into a class `MyClass` and a function `my_function()`. The scripted portion of the CLI only takes user input, calls `MyClass` and `my_function()`, and outputs the final value of the workflow. You may find it necessary at times to include `for` loops, `while` loops, and `if/elif/else` statements in this portion of the CLI, but make sure to separate and organize your code into classes and functions whenever possible.

To make this even clearer, you can separate related functions and classes into different files and **import** them into your CLI script.

Validate User Input

You may have noticed in our `grade_reports` CLI that we did *not* validate user input. If you wanted to enter a number or a nonsensical string, not only could you have done so, but it would have been written to `grade_report.txt` !

A good CLI will check the format of user input before using it to perform any actions. This can be carried out using regular expressions through the `re` module [↗\(https://docs.python.org/3/library/re.html\)](https://docs.python.org/3/library/re.html), the built-in `type()` and `isinstance()` functions, the various Python operators, and more.

► *What two arguments does `isinstance()` take?*

Keep the End User Informed

You may have noticed while using Pipenv that many things are printed to the command line while external libraries are added to your virtual environment. This lets you know that Pipenv is doing its job. Most importantly, Pipenv finishes with a message as to what was installed, where it was installed, and how to access it later on.

Compare this with our `grade_reports` CLI. What do we see when the script finishes execution? We can find our new file in the `reports` directory, but how would someone who downloaded your CLI from PyPI know that?

A CLI should always inform the end user of what it is doing and where to find output. This should always happen through the CLI itself, but it's often helpful to store the messages from a session in a `logs` directory as well.

Use External Libraries to Standardize Your Code

While we at Flatiron encourage you to show us as much of your amazing hand-written code as possible, it's often best to use popular external libraries to handle common programming tasks. This makes it easier to "onboard" new developers to your application if you ever need to hire a team to maintain it.

Two popular libraries that will help you make amazing CLIs are `Click`  (<https://click.palletsprojects.com/en/8.1.x/>) and `Fire`  (<https://google.github.io/python-fire/guide/>). While we won't require you to use these in your Phase 3 project, we strongly recommend that you use one. (It's another library to add to your résumé!)

Conclusion

CLIs are a helpful tool for managing the data on your computer and on remote servers. A well-built CLI will allow you to let non-technical users complete many of the same tasks with your application that you can from the Python shell without very much training at all. Consider the best practices for building CLIs as you work through the Phase 3 project this week- make sure to look back at the earlier modules, too!

Resources

- [Click documentation](https://click.palletsprojects.com/en/8.1.x/)  [\(https://click.palletsprojects.com/en/8.1.x/\)](https://click.palletsprojects.com/en/8.1.x/)
- [The Python Fire Guide](https://google.github.io/python-fire/guide/)  [\(https://google.github.io/python-fire/guide/\)](https://google.github.io/python-fire/guide/)